FINALSDP / 01

# GiftBox Studio

## A configurable gift-box application

Software Design Patterns / Final Project Report


| Team member | Group |
| --- | --- |
| Saparali Shalkar | SE-2532 |
| Nurbayev Murager | SE-2532 |
| Mukhametov Temirlan | SE-2532 |


Repository: https://github.com/Temirlan222/FinalSDP

Report date: 11 November 2026. The repository is private; the lecturer must be granted access.

## Introduction

GiftBox Studio is a Java application for assembling a gift box. A customer picks gifts, adds wrapping or a ribbon, and selects a discount. After creating an order, the customer can move it through packing to ready for collection. The browser shows the price and the status messages returned by Java.

The project uses six design patterns: Factory Method, Builder, Decorator, Strategy, Observer and Facade. The gift-box example gives each one a separate job. Adding tea to the catalogue concerns product creation; adding a ribbon concerns one existing gift. A new discount changes the price calculation, while a new subscriber changes who receives an order update.

## What this submission contains

The repository contains the Java application, its HTML/CSS/JavaScript interface, build scripts and tests. The report is accompanied by editable UML files and screenshots. Java calculates every price and validates order changes. The browser sends the selected options and displays the response.

This is a local coursework application. It has no accounts, payment processing, delivery service or email connection. Orders are stored in memory and disappear when the Java process stops.

# Problem, scope and architecture

## Functional requirements

A box contains between one and twelve gifts. The catalogue has a book for 1500 KZT, a candle for 800 KZT and a tea collection for 1200 KZT. Wrapping adds 200 KZT to an item; a ribbon adds 100 KZT. The recipient name is required and the message is optional. One discount applies to the total after decorations have been added.

The order sequence is CREATED, PACKING, then READY. Each valid change sends an update to the customer and packing-desk subscribers. The application rejects a skipped or repeated stage. It also rejects empty boxes and unknown products before creating an order.

| Layer | Responsibility | Main files |
| --- | --- | --- |
| Browser | Selection, accessible controls, responsive layout, display of Java results | web/index.html, style.css, app.js |
| HTTP adapter | Bounded form input, routing, JSON responses, local static assets | web/WebServer.java, Json.java |
| Application | Compose creation, decoration, building, pricing and order creation | facade/GiftBoxFacade.java |
| Domain | Products, wrappers, policies, box and order events | api, factory, builder, decorator, strategy, pricing, observer |

## Dependencies and runtime

The application requires JDK 11 or later. It uses the HTTP server included in the JDK and listens on 127.0.0.1. There are no external Java libraries, and the browser uses standard JavaScript APIs. The gift illustrations are CSS shapes stored with the project, so the demonstration can run offline once the files are available.

## Data and validation

Prices are whole KZT values stored as nonnegative integers. Percentage discounts multiply using a long value before rounding the final total down. Math.addExact checks for overflow when prices are added. GiftBox.Builder trims text, checks the 60-character recipient and 200-character message limits, and copies the gift list into the finished box.

The finished list cannot be changed, but it still contains references to the original gifts. Built-in products and decorators are immutable. Any additional Gift implementation must also keep its price and description stable after it is added to a box. CONTRACT.md records this requirement.

# Creating gifts and building a box

## Factory Method

GiftCreator defines the public createGift method and the protected factory method createProduct. Calling createGift runs the subclass implementation of createProduct, then validates the result. BookCreator, CandleCreator and TeaCreator each return their own Gift product. A product provides its description and base price; decoration and discounts are handled elsewhere.

```
public final Gift createGift() {
    Gift gift = createProduct();
    if (gift == null || gift.getPrice() < 0 || gift.getDescription().isBlank()) {
        throw new IllegalStateException("The creator returned an invalid gift");
    }
    return gift;
}
protected abstract Gift createProduct();
```

The factory method is in factory/GiftCreator.java, with overrides in the three creator classes. GiftBoxFacade selects one of these creators using a map. The map is a lookup mechanism; the subclass override is the Factory Method relationship. ProjectTests checks that an additional creator can be supplied without changing the facade algorithm.

## Builder

GiftBox.Builder accepts the recipient, an optional message and one gift at a time. Its build method checks the completed selection and returns a GiftBox. The box copies the list, so adding another gift to the builder later does not change a box that already exists. PriceCalculator handles the price after construction.

```
GiftBox.Builder builder = new GiftBox.Builder()
        .forRecipient(recipient).withMessage(message);
for (GiftSelection selection : selections) {
    builder.addGift(createDecoratedGift(selection));
}
GiftBox box = builder.build();
```

This excerpt follows GiftBoxFacade.quote. The production loop also rejects a null selection. builder/GiftBox.java contains the builder; the tests cover empty input, text and item limits, an unmodifiable result list and builder reuse.

## Why both patterns are useful

The two creation patterns operate at different levels. A creator returns one product, such as a candle. The builder collects products into a box and validates the box as a whole. In a request, the facade creates each gift, adds any decorations, and passes it to the builder. Optional text and a variable number of gifts make these steps easier to read than one long constructor call.

# Decoration and pricing

## Decorator

Gift declares getDescription and getPrice. GiftDecorator implements the same interface and holds another Gift. GiftWrap and Ribbon call the wrapped object and add their own description and price. Since they accept the interface, the same decorators work with a book, candle or tea product.

```
Gift decorated = new Ribbon(new GiftWrap(book));
// GiftWrap.getPrice():
return Math.addExact(super.getPrice(), 200);
// Ribbon.getPrice():
return Math.addExact(super.getPrice(), 100);
```

For a 1500 KZT book, the chain above reports 1800 KZT: 1500 + 200 + 100. The original book still reports 1500. There is no need for a separate BookWithRibbonAndWrap class. Swapping these wrappers changes the description order but leaves the price the same. The domain API permits repeated wrapping; the interface offers each decoration once per item.

## Strategy

PriceCalculator holds a DiscountStrategy and calls applyDiscount after adding the decorated gift prices. setDiscountStrategy can replace the policy at runtime. Four policies are available: NoDiscount, StudentDiscount, HolidayDiscount and ThresholdDiscount.

| Policy | Rule | Example |
|---|---|---|
| NoDiscount | Return subtotal | 2600 → 2600 |
| StudentDiscount | floor(subtotal × 90 / 100) | 2600 → 2340 |
| HolidayDiscount | floor(subtotal × 80 / 100) | 2600 → 2080 |
| ThresholdDiscount | Subtract 500 when subtotal ≥ 5000 | 4999 → 4999; 5000 → 4500 |

```
public int applyDiscount(int subtotal) {
    if (subtotal < 0) {
        throw new IllegalArgumentException("Subtotal cannot be negative");
    }
    return subtotal >= MINIMUM_SUBTOTAL ? subtotal - SAVING : subtotal;
}
```

StudentDiscount and HolidayDiscount use percentages. ThresholdDiscount follows a different rule: it subtracts 500 KZT only when the subtotal reaches 5000 KZT. PriceCalculator calls both kinds through the same interface. Adding another policy leaves the calculator unchanged, although the HTTP adapter and interface still need to expose the new option.

# Orders, events and the facade

## Observer

Order keeps a list of OrderObserver subscribers. CustomerNotifications and PackingDesk implement onStatusChanged and each store their own message history. subscribe prevents duplicate registrations and unsubscribe removes a listener. During a status change, Order calls the subscribers from a copy of the list.

```
public void advanceTo(OrderStatus next) {
    Objects.requireNonNull(next, "status");
    if (next.ordinal() != status.ordinal() + 1) {
        throw new IllegalArgumentException("Follow CREATED -> PACKING -> READY");
    }
    status = next;
    for (OrderObserver observer : List.copyOf(observers)) {
        observer.onStatusChanged(id, status);
    }
}
```

The ordinal check relies on the order of the enum constants. It supports the current three-stage sequence; cancellation or branching would need an explicit set of allowed transitions. Callbacks run synchronously and save messages in memory. The browser receives the updated histories in the HTTP response. No email or push service is involved.

## Facade

The HTTP layer calls GiftBoxFacade.quote with the recipient, message, selected gifts and discount strategy. The facade chooses creators, wraps gifts, builds the box and asks PriceCalculator for the total. placeOrder uses that quote to create an OrderReceipt. The receipt creates an Order and attaches the two subscribers. Requesting a quote alone creates no order and sends no notifications.

```
Quote quote = quote(recipient, message, selections, strategy);
return new OrderReceipt("GB-" + UUID.randomUUID(), quote);
```

GiftBoxFacade.java contains the assembly steps. OrderReceipt.java connects the order to its subscribers, and observer/Order.java handles events. The HTTP layer therefore needs a single application entry point, while validation, arithmetic and event dispatch remain in their respective classes.

## One coordinated scenario

Creating one decorated box uses Factory Method, Decorator, Builder, Strategy and Facade. Moving that order to PACKING or READY then triggers Observer. The patterns participate in the same order lifecycle, which is the coordinated scenario required by the assignment.

# Principles and Clean Code decisions

| Principle | Concrete evidence and consequence |
| --- | --- |
| Single Responsibility | GiftWrap changes presentation; PriceCalculator calculates money; GiftBox.Builder assembles a box; WebServer translates HTTP. Changing an HTTP response does not require editing a discount algorithm. |
| Open/Closed | A new DiscountStrategy works with PriceCalculator unchanged. A new GiftCreator can be injected into GiftBoxFacade. UI options and default registration still need updating. |
| Liskov Substitution | Products and decorators provide the same Gift operations and nonnegative prices. PartTests uses an independent Gift implementation to check that decorators and pricing do not depend on a particular factory. |
| Interface Segregation | Gift has two operations, DiscountStrategy one, OrderObserver one. Pricing does not depend on order status methods or browser controls. |
| Dependency Inversion | PriceCalculator depends on DiscountStrategy and Gift. Order depends on OrderObserver. The facade accepts a creator map; its convenience constructor wires concrete defaults at the composition boundary. |
| DRY and readable names | DiscountMath centralizes percentage validation and safe multiplication. createDecoratedGift, calculateSubtotal and advanceTo communicate intent. Shared form parsing and JSON output are isolated in the web layer. |

## Defensive behavior

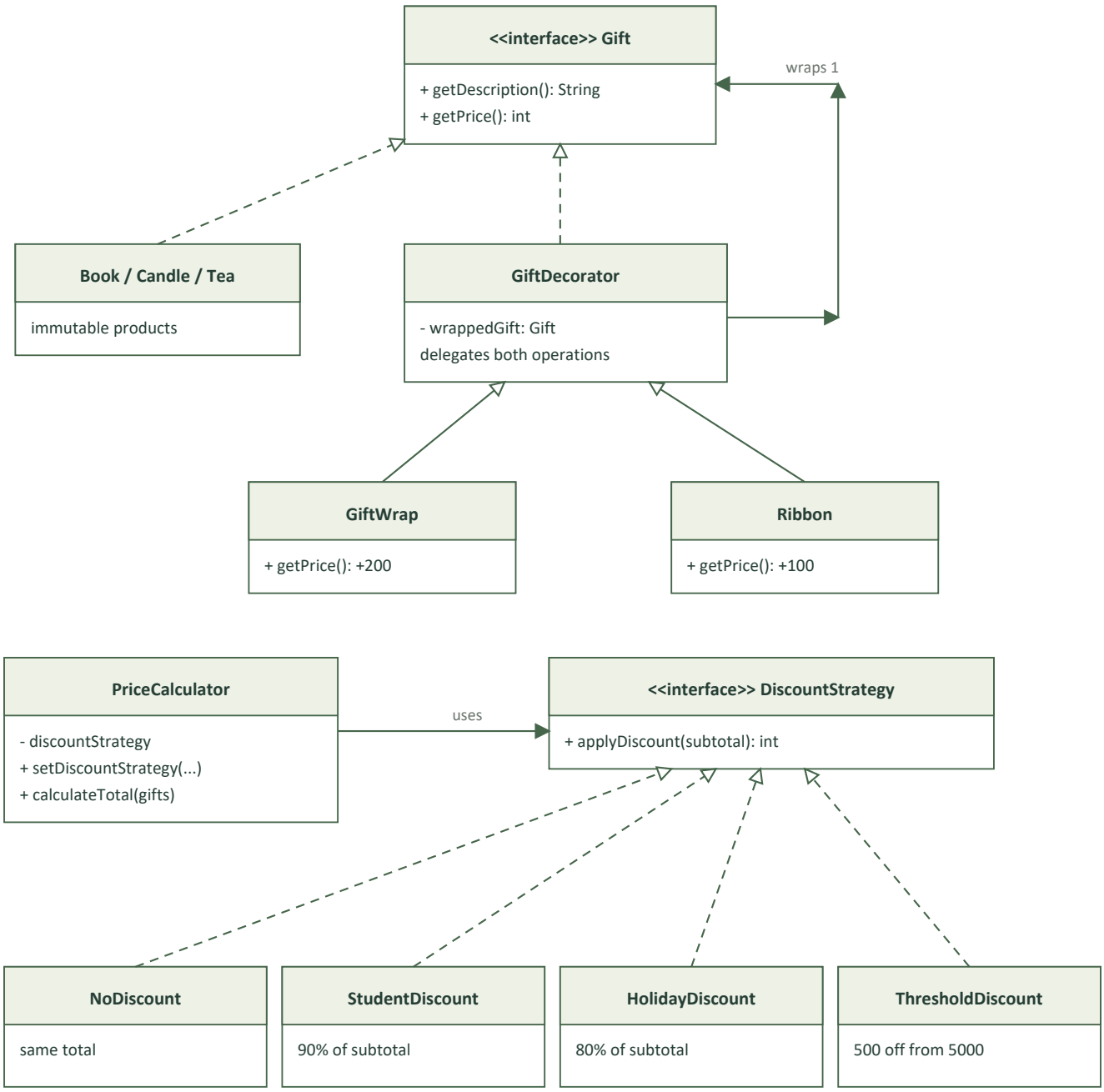
Validation is placed beside the rule it protects: the builder checks box contents and the calculator checks amounts. The HTTP layer limits request size, rejects unsupported methods and origins, and serves a fixed set of local files. In the browser, user text is displayed with textContent. Changing a selection clears the old quote, and an older response cannot overwrite a newer result.

## Implementation limits

The JSON helper only writes responses; it is not a general parser. This keeps external libraries out of the project. The server handles requests sequentially on the local computer. Subscriber failures are another limitation: a thrown callback exception is not isolated, so an unreliable external subscriber would need error handling before it could be used safely.

# UML: Decorator and Strategy

Simplified class diagrams. A hollow triangle points to the parent or interface; a dashed line denotes interface implementation. Plain arrows show associations or use. Methods not needed for the pattern are omitted.

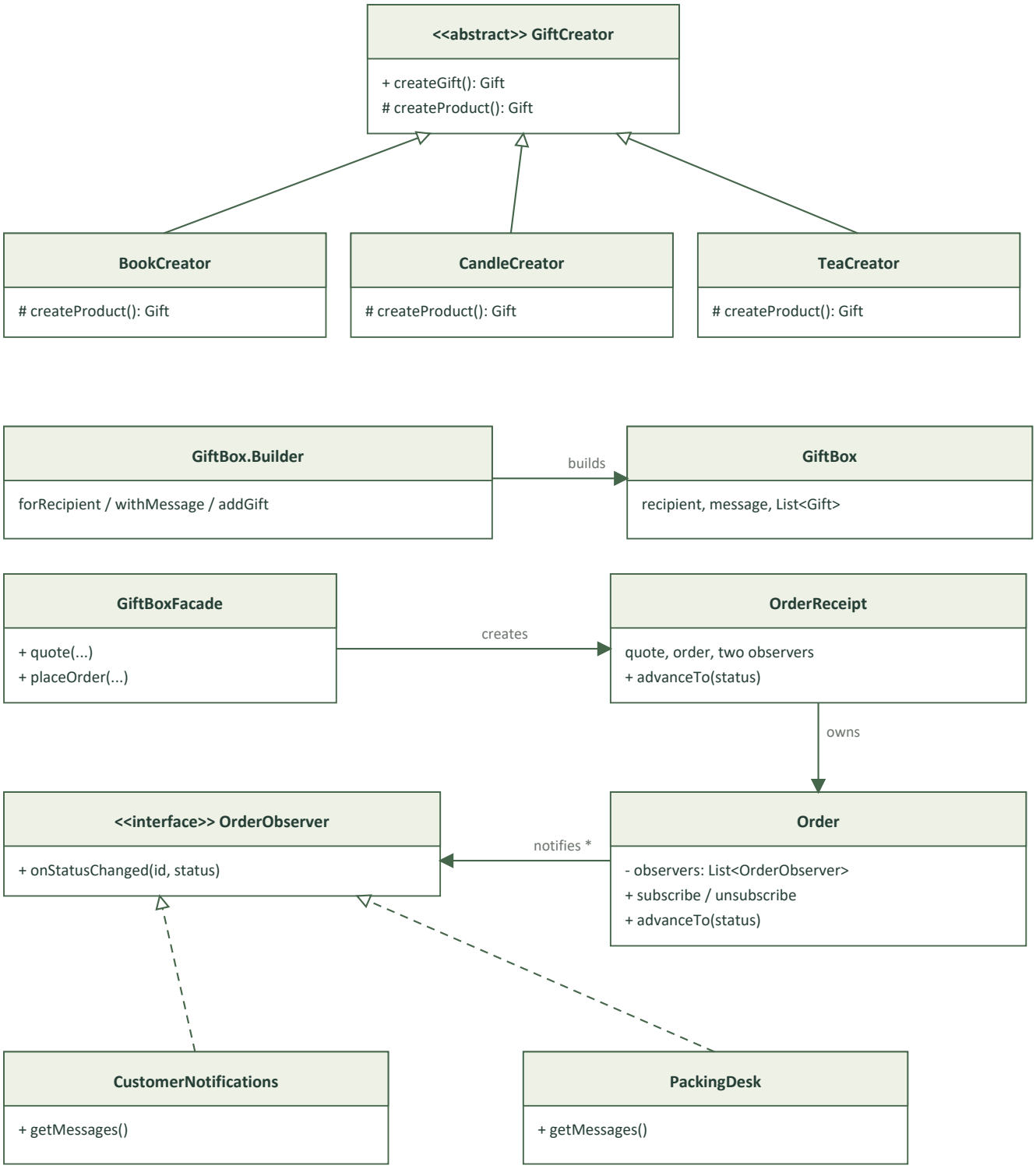



## How to read the relationships

GiftDecorator implements Gift and contains a reference to another Gift. Following that reference leads through the wrappers to the product. PriceCalculator instead contains a DiscountStrategy reference. Replacing it changes the discount rule while leaving the gifts and subtotal calculation in place.

Editable sources: docs/uml/classes.puml. Separate vector versions: decorator.svg and strategy.svg. Actual source: api, decorator, pricing and strategy packages.

# UML: creation and order coordination

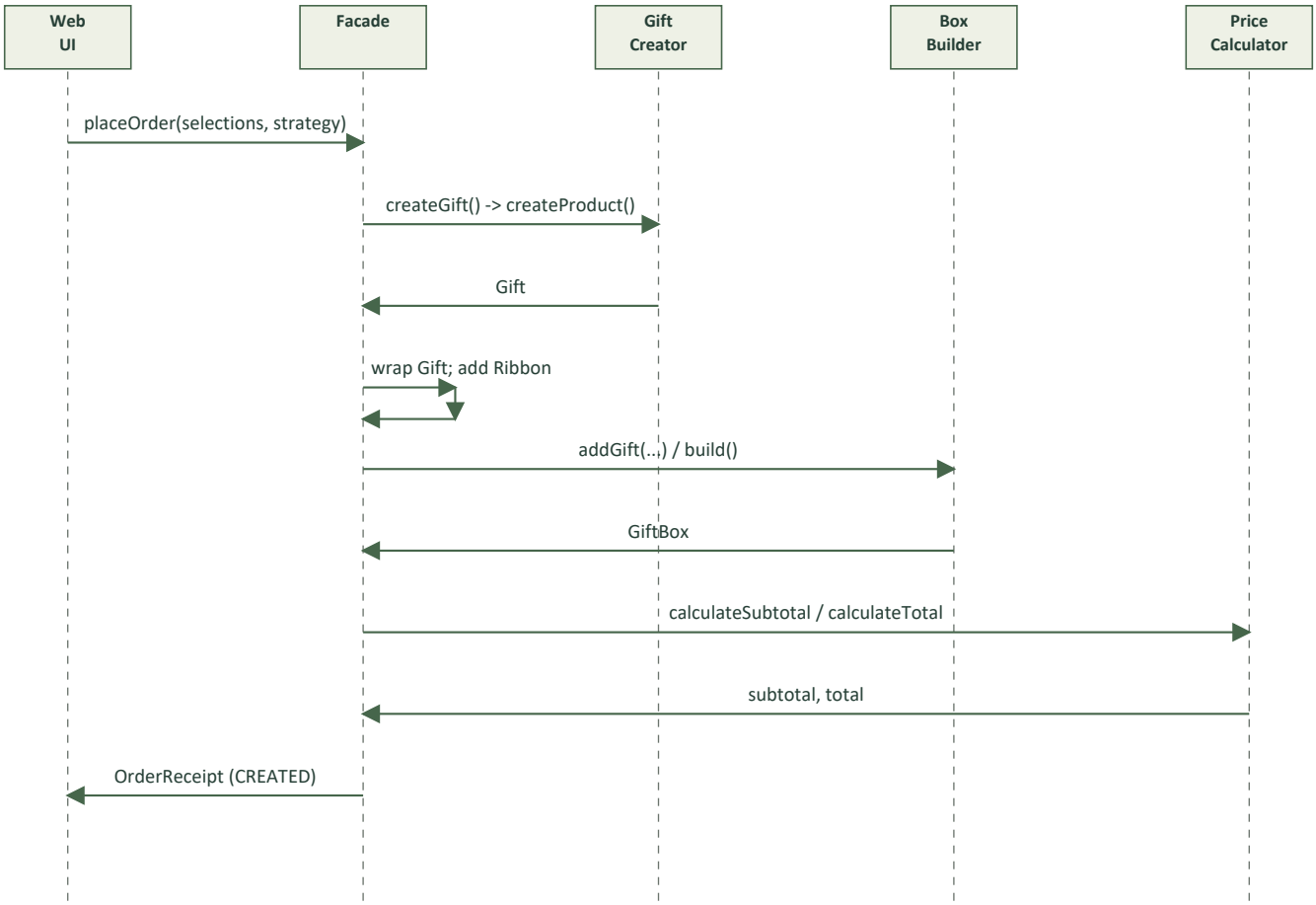


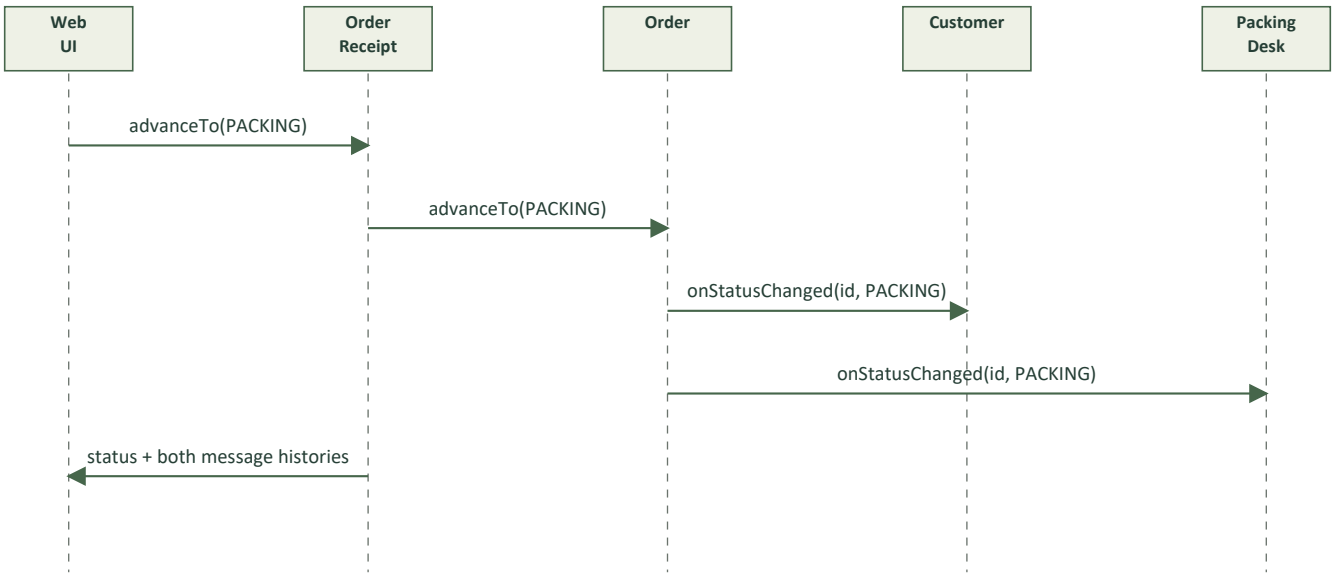

Each concrete creator overrides createProduct. GiftBox.Builder collects the returned gifts into a GiftBox. GiftBoxFacade coordinates those types with the decorators and PriceCalculator shown on page 7, then creates an OrderReceipt when an order is placed.

OrderReceipt subscribes CustomerNotifications and PackingDesk to its Order. Order dispatches through the OrderObserver interface, so its event loop contains no separate calls for customer and packing-desk classes. These associations match the Java implementation.

# UML: request and event sequences

Web/UI represents the browser and HTTP adapter together. The first sequence follows placeOrder through the Java classes. createGift calls the subclass factory method. A quote request uses the same gift-building and pricing steps but stops before order creation.




The second sequence shows the move to PACKING. Moving to READY calls the same two observers and adds another entry to each history. A skipped stage is rejected before notification. The editable sequence diagram is in docs/uml/sequence.puml.

# Application screenshots

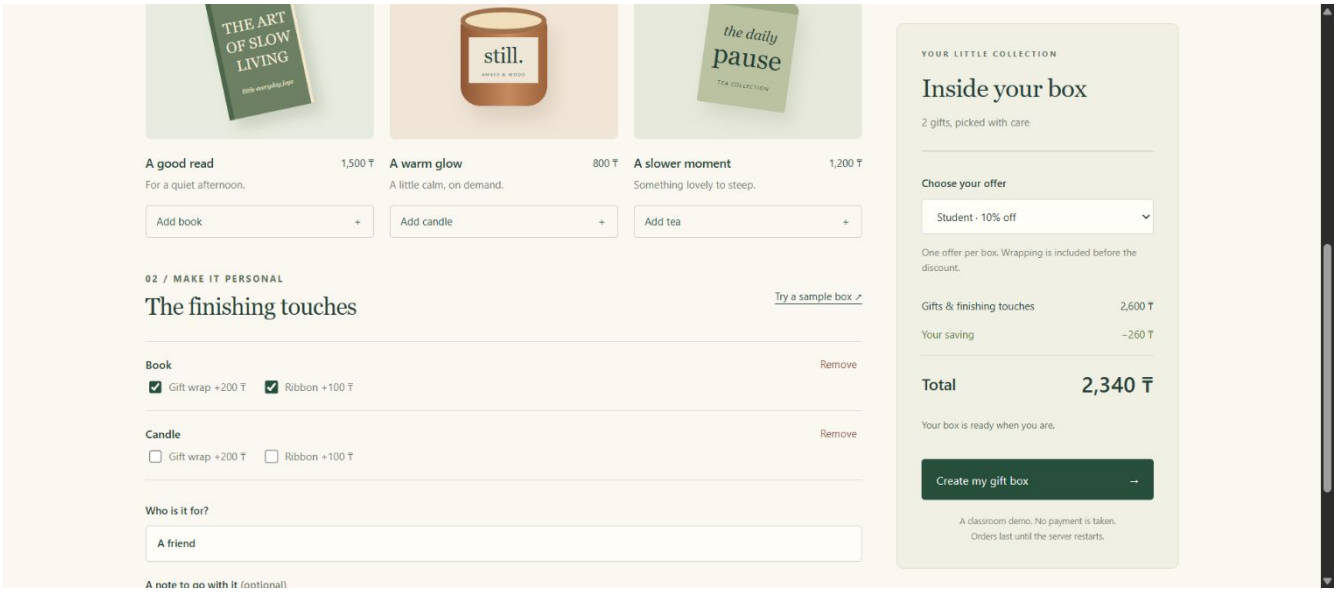



**Figure 1.** The sample contains a book with wrapping and ribbon, plus a candle. With the Student offer selected, Java returns a subtotal of 2600 KZT and a total of 2340 KZT. Screenshot: 01-builder.jpg.

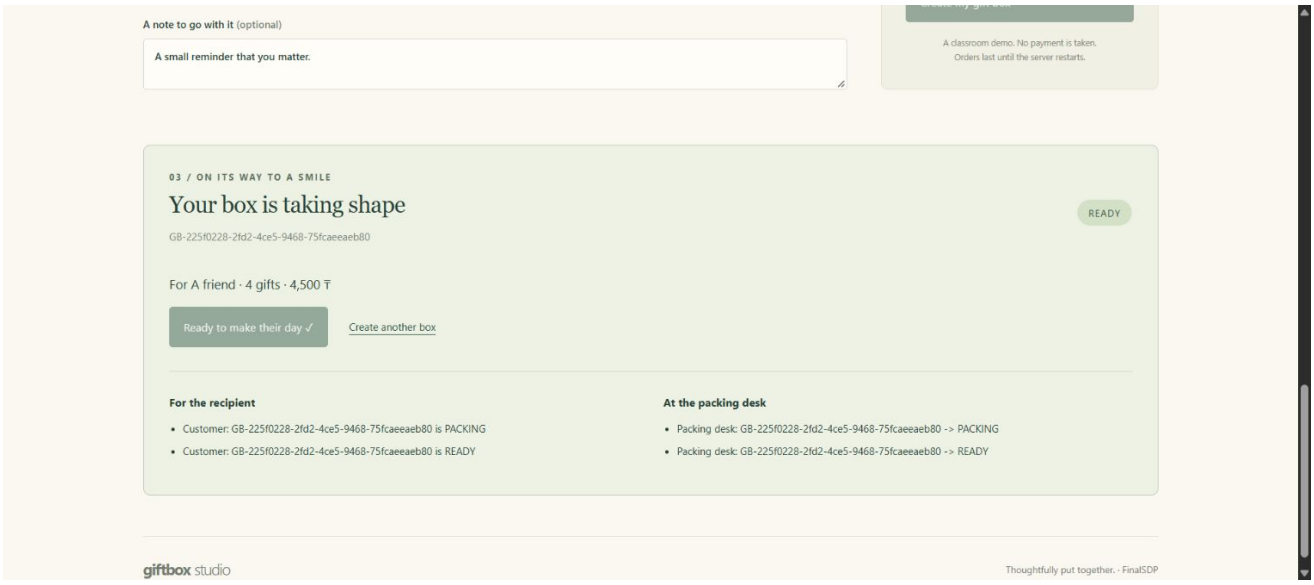



**Figure 2.** The four-gift threshold example reaches 5000 KZT and receives a 500 KZT discount. The order has moved from CREATED through PACKING to READY, and both message histories contain the two updates. Screenshot: 03-order-ready.jpg.

These images were captured from the running application. docs/screenshots also contains the threshold and mobile views. The mobile check used a 390 px viewport, with 375 px available beside the scrollbar; the page had no horizontal overflow.

# Verification and reproducibility

| Check | Result | Evidence |
|---|---|---|
| Pricing and decoration | 12 passed / 0 failed | test/giftbox/PartTests.java |
| Full project | 12 passed / 0 failed | test/giftbox/ProjectTests.java |
| HTTP integration | 20 passed / 0 failed | test/http_smoke.py |
| Browser workflow | Verified interactively | Sample pricing, threshold, order creation, PACKING, READY, two histories |
| Responsive layout | 390 px viewport checked | Content width matched viewport content width; mobile screenshot |
| JavaScript syntax | Passed | Node --check web/app.js |

## Important test cases

PartTests covers decoration chains, unchanged base gifts and an independent Gift implementation. It also checks strategy replacement, applying a discount once per box, rounding, empty lists, nulls, negative values, invalid strategy results and overflow. ProjectTests covers creator extension, builder snapshots, text and item limits, the discount threshold, unsubscribe, rejected transitions and the full order scenario.

The HTTP suite sends requests to the running Java server. It checks valid quotes, JSON escaping, order creation and both notification histories. Error cases include invalid transitions, unknown products or offers, empty input, too many gifts, missing orders, unsupported methods, oversized bodies and rejected origins. These tests cover the local demonstration; they do not measure multi-user capacity or establish production security.

## Run from the source archive

```
# Windows PowerShell, from the extracted project folder
powershell -ExecutionPolicy Bypass -File ./build.ps1 -Test -Run
powershell -ExecutionPolicy Bypass -File ./build.ps1 -Web
# Open http://127.0.0.1:8080
# In a second terminal, with Python 3:
python test/http_smoke.py
```

The recorded checks used Windows, JDK 25 with --release 11, and Chrome. Execution on JDK 11 itself and Linux/macOS was not checked. Python is needed for the optional HTTP tests only; the application runs without it.

# Conclusion and further work

## Conclusion

The application covers the full demonstration: select gifts, add decorations, calculate a discount, place an order and view its progress. Each of the six patterns has a role in that sequence. The tests cover the main path and boundaries such as a 4999 versus 5000 KZT subtotal or an attempted jump straight to READY.

Gift contents, pricing and order events can change separately. For example, the threshold discount uses the same calculator as a percentage discount, and a new observer can receive events without changing the price calculation. The code is small enough to follow a browser action from the HTTP handler into the relevant domain classes during the defense.

## Further work

| Improvement | Reason and affected area |
|---|---|
| Persistent orders | Add a repository and storage so restarting does not lose orders. Keep persistence outside pricing and decoration. |
| Reliable notifications | Isolate observer failures and, if needed, enqueue external delivery. Add failure tests before sending real email. |
| Richer workflow | Replace enum-order validation with explicit allowed transitions if cancellation or delivery branches are introduced. |
| Production deployment | Add identity, authorization, concurrency controls, durable storage and operational monitoring before exposing the server publicly. |
| Catalogue extension | Implement another GiftCreator and add its UI card. Keep the Gift interface stable and test decoration compatibility. |

## Team responsibility and defense

Responsibilities are assigned as follows: Mukhametov Temirlan covers Strategy and Decorator; Saparali Shalkar covers Factory Method and Builder; Nurbayev Murager covers Observer and Facade. The team may exchange the latter two assignments. These are preparation and development responsibilities, not a verified record of individual authorship.

AI assistance was used to prepare the implementation. The course also requires each student to implement at least two patterns individually. The files alone do not establish that this requirement has been met; each member must develop, understand and defend their own contribution under the course rules.

## References and submission sources

1. Final Project Defense Requirements.pdf, provided course handout, page 1 (requirements).
2. Instructor upload message: source ZIP/RAR, report with GitHub link and at least six pages, additional UML/schemas.
3. Delivered Java sources, CONTRACT.md, TEAM.md and test suites. Repository link appears on page 1.